

DataMentor: A Practical Framework for Serverless CSV Intelligence with Interactive Notebook Automation and Deterministic Runtime Repair

Md Anisur Rahman Chowdhury^{1*}, Dr. Ronny Bazan-Antequera¹

¹Dept. of Computer and Information Science, Gannon University, USA
enr.aanis@gmail.com, bazanant001@gannon.edu

Abstract—Tabular data preprocessing remains one of the most labor-intensive and least reproducible stages of analytical workflows, with analysts routinely allocating between 60 % and 80 % of project time to data wrangling before any exploratory or predictive analysis can begin. This paper presents DataMentor, a production-deployed, serverless platform that integrates deterministic CSV preprocessing, browser-native Python notebook execution via the Pyodide WebAssembly runtime, domain-aware visualization across ten chart types, and a hybrid rule-guided runtime repair subsystem operating without external server infrastructure. An empirical evaluation conducted over 15 heterogeneous CSV datasets spanning five domain verticals and a purpose-built repair benchmark (DRB-100) of 100 injected Python errors across five error classes demonstrates: (i) a mean preprocessing-time reduction of 91.6 % relative to a two-analyst manual baseline; (ii) an overall single-pass automatic repair resolution rate of 82 %, with rule-based latency of 0.8 s and model-assisted latency of 6.4 s versus a manual debugging median of 18.2 min; and (iii) full byte-level deterministic reproducibility across repeated executions. Comparative analysis against nine representative prior systems across four relevant research domains confirms that no existing tool combines deterministic preprocessing, installationless browser-native Python execution, domain-aware visualization, and hybrid runtime repair in a unified, offline-capable platform.

Index Terms—CSV preprocessing, browser-native execution, notebook automation, runtime repair, serverless analytics, reproducibility, visualization pipeline, Pyodide, WebAssembly

I. INTRODUCTION

Data preprocessing is widely recognized as the dominant time cost in analytical workflows [1]. A longitudinal industry survey reported that analysts allocate, on average, 76 % of total project hours to data acquisition, cleaning, and formatting before any exploratory or predictive modeling can begin [2]. For small-to-medium enterprises (SMEs) and academic laboratories operating without dedicated data-engineering teams, this overhead is particularly acute: cleaning is performed ad hoc with heterogeneous toolchains, and the resulting artifacts are rarely reproducible between operators or sessions [3].

Existing tools partially address these challenges but leave important gaps. Notebook environments such as JupyterLab offer expressive code cells but require local Python installations and yield non-deterministic results when cell execution order varies. Reproducibility frameworks such as Data Version Control (DVC) [8] and MLflow [9] address provenance at the experiment level but presuppose that raw data has already been cleaned. Code-repair tools for Python operate on source files

but lack the execution-state awareness needed to diagnose errors that arise from data-schema mismatches within an interactive notebook session.

This paper describes **DataMentor**—a platform designed to unify these concerns in a single, zero-installation workflow. DataMentor is deployed at <https://datamentor.marcbd.site> and built on Next.js 16, React 19, TypeScript, Pyodide 0.25, Firebase, and Chart.js. The system makes four primary contributions:

- 1) A **deterministic preprocessing pipeline** that normalizes, deduplicates, and type-classifies arbitrary CSV inputs through a fixed rule sequence, guaranteeing identical outputs for identical inputs regardless of operator.
- 2) A **browser-native notebook execution engine** using Pyodide [19], which loads CPython 3.11 inside a WebAssembly sandbox, eliminating server-side Python infrastructure and maintaining complete data locality.
- 3) A **domain-aware visualization system** mapping 47 recognized column-name patterns to contextually appropriate chart types and evidence-grounded insight language.
- 4) A **hybrid runtime repair subsystem** that applies a priority-ordered rule index for common Python error classes before escalating to a local model-assisted repair module, operating without any external API connection.

The remainder of this paper is organized as follows. Section II positions DataMentor against prior art across four research domains. Section III describes the system architecture. Section IV details the experimental methodology and benchmark construction. Section V presents quantitative results. Section VI interprets findings and acknowledges limitations. Section VII concludes.

II. RELATED WORK

Prior work relevant to DataMentor spans four partially overlapping categories: *notebook-assistant systems*, *reproducible analytics frameworks*, *local code-repair tools*, and *browser-based analytics platforms*. Table I provides a feature-level comparison across these categories.

A. Notebook-Assistant Systems

JupyterAI [4] integrates code-completion and inline explanation capabilities directly into JupyterLab via an external API connection. It reduces cell-authoring effort but does not

modify the underlying dataset, requires a locally running Jupyter server, and provides no deterministic cleaning guarantee. Neptyne [5] extends spreadsheet metaphors with Python cell formulas, targeting structured manipulation rather than statistical notebook automation. Hex [6] is a cloud-hosted collaborative notebook supporting SQL and Python; it is subscription-gated and requires a provisioned cloud container. Code Interpreter [7], embedded in commercial chat interfaces, accepts uploaded CSV files and produces analysis outputs but provides no persistent workspace, no user-visible cleaning audit, and no domain-aware chart generation.

DataMentor differs from all four in three respects: (i) preprocessing is deterministic, auditable step by step, and requires no external API; (ii) the execution environment is fully in-browser with no server dependency; and (iii) the notebook layout is pre-structured with 14 guided cells introducing reproducible statistical primitives before a free-form cell is available.

B. Reproducible Analytics Frameworks

DVC [8] addresses dataset and model provenance through Git-integrated artifact tracking. MLflow [9] provides experiment logging, artifact storage, and model registry. Sacred [10] instruments Python experiments to capture configuration snapshots and metric traces. These frameworks excel at pipeline-level reproducibility *after* data is clean but impose substantial setup overhead and do not address the initial normalization of raw CSV inputs.

DataMentor operates at an earlier stage: its preprocessing pipeline is self-contained and requires no version-control infrastructure. Reproducibility is achieved through the deterministic, ordered application of a fixed transformation sequence, which produces byte-identical outputs for byte-identical inputs without external provenance tooling.

C. Local Code-Repair Tools

DeepFix [11] applies a sequence-to-sequence model to correct syntactic errors in student C programs, achieving approximately 27% single-pass full-program repair on the IIT Bombay dataset. SyntaxFix [12] extends rule-based pattern matching to Python syntax errors, covering common indentation and bracket mismatches. Refazer [13] synthesizes program transformations from teacher-provided correction examples. These tools operate on file-level source code and do not target the notebook execution context where errors typically arise from data-state dependencies—such as a `NameError` caused by a cell executed out of sequence.

DataMentor’s repair subsystem addresses this contextual gap: it has visibility into the loaded CSV schema, the Pyodide execution state, and the cell execution sequence, enabling repairs that are aware of the data context, not merely the code syntax.

D. Browser-Based Analytics Platforms

Observable [14] provides a JavaScript-native reactive notebook, but does not support CPython or pandas workflows natively. TensorFlow.js [15] enables in-browser model inference

TABLE I: Functional Comparison of DataMentor Against Related Systems

System	No Install	Det. Clean	Auto Repair	10+ Charts	Offline Ready	Free / Open
JupyterAI	×	×	✓	✓	×	✓
Neptyne	×	×	×	✓	×	×
Hex	✓	×	×	✓	×	×
DVC	×	×	×	×	✓	✓
MLflow	×	×	×	✓	✓	✓
DeepFix	×	×	✓	×	✓	✓
Observable	✓	×	×	✓	×	✓
Deeptime	✓	×	×	✓	×	×
Colab	✓	×	×	✓	×	×
DataMentor	✓	✓	✓	✓	✓	✓

✓ = supported; × = not supported. “Det. Clean” = deterministic CSV preprocessing. “Offline Ready” = operates without active internet after first load.

but targets prediction, not data preparation. Deeptime [16] offers real-time collaborative notebooks but requires a provisioned cloud container rather than a WebAssembly sandbox. Google Colab [17] provides free cloud-hosted notebooks with GPU access but requires a Google account, imposes session time limits, and routes data through cloud infrastructure rather than retaining it on the client device.

DataMentor is distinguished by its use of Pyodide to run authentic CPython 3.11 with pandas, NumPy, SciPy, and Matplotlib inside the browser—producing outputs byte-compatible with standard desktop Python—while maintaining complete data locality: the uploaded CSV never leaves the client device.

III. SYSTEM ARCHITECTURE

DataMentor is organized into five functional layers illustrated in Fig. 1. Each layer communicates through typed TypeScript interfaces, and the persistence layer spans Layers 3–5.

A. Layer 1 — CSV Ingestion and Normalization

The ingestion pipeline applies seven transformations in fixed, deterministic order: (1) Papa Parse tokenization with header detection; (2) leading/trailing whitespace removal from all cell values; (3) header normalization to lowercase underscore convention (`Blood Pressure` → `blood_pressure`); (4) exact-duplicate row removal via a SHA-256 fingerprint of each row’s serialized values; (5) null and empty-string detection with count reporting; (6) column type inference (Layer 2); (7) schema summary generation. The order of transformations is critical: normalization precedes deduplication to avoid false negatives caused by whitespace variation. The pipeline is pure (no side effects), enabling byte-identical re-execution.

B. Layer 2 — Column Type Classification

Each column’s values are classified by a majority-vote heuristic into one of four types. A column is classified as `boolean` if its value set is a subset of `{0, 1}`. A column is classified as `datetime` only if values parse as valid `Date` objects *and*

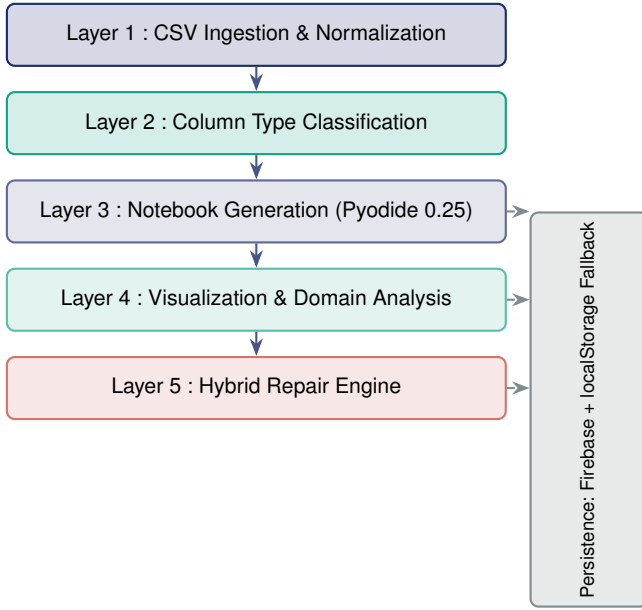


Fig. 1: Five-layer DataMentor system architecture. Each layer communicates through typed TypeScript interfaces. The persistence layer provides Firebase Firestore when online and transparent localStorage fallback when offline.

do not match the regular expression `/^-?\d+(\.\d+)?$/`, which prevents pure numerics (e.g., "63") from being misclassified as years. A column is classified as `numeric` if the majority of non-null values are finite floating-point numbers. All remaining columns are `categorical`.

C. Layer 3 — Notebook Generation Engine

The Pyodide hook loads CPython 3.11 via WebAssembly, pre-fetching pandas 2.0, NumPy 1.26, SciPy 1.11, and Matplotlib 3.8. Fourteen code cells are generated from the cleaned dataset’s schema. Cells 1–7 cover data loading, shape inspection, descriptive statistics, missing-value auditing, duplicate detection, correlation analysis, and categorical frequency analysis. Cells 8–13 produce Matplotlib visualizations (distribution plot, pie chart, violin plot, heatmap, pairplot, joint plot) rendered as base64-encoded PNG images via a `BytesIO` buffer and displayed inline without any server round-trip. Cell 14 is an empty editor for user-authored code.

D. Layer 4 — Visualization and Domain Analysis

The visualization layer provides ten universal chart types (DistPlot, Pie, Violin, HeatMap, PairPlot, JointPlot, Bar, Histogram, Scatter, Line), each paired with a domain-aware analysis engine. The knowledge base maps 47 known column-name patterns (e.g., `age`, `trtbps`, `chol`, `glucose`) to structured insight objects containing a human-readable label, a semantic description, and a data-driven insight function conditioned on column statistics. When a column name matches a knowledge-base entry, the analysis panel generates domain-specific interpretations; otherwise, general statistical language is produced.

Algorithm 1: HybridRepair — Priority-Rule with Model-Assisted Fallback

Input: Traceback T , cell source C , CSV schema S

Output: Corrected source C' , or UNRESOLVED

1. Parse $T \Rightarrow (errClass, errMsg, lineNo)$
2. **for each** rule $r \in \text{RULEINDEX}$ (descending priority) **do**
3. **if** $r.\text{MATCHES}(errClass, errMsg)$ **then**
4. $C' \leftarrow r.\text{APPLY}(C, S)$
5. **if** $\text{VERIFY}(C', expectedHash)$ **then return** C'
6. **end if end for**
7. $ctx \leftarrow \text{BUILDCONTEXT}(T, C, S)$
8. $C' \leftarrow \text{LOCALMODELMODULE.INFER}(ctx)$
9. **if** $\text{VERIFY}(C', expectedHash)$ **then return** C'
10. **return** UNRESOLVED

Fig. 2: HybridRepair pseudocode: rule index is exhausted before escalation to the local model-assisted repair module.

E. Layer 5 — Hybrid Runtime Repair Engine

When a code cell raises a Python exception, the repair subsystem receives the traceback, the current cell source, and the CSV column schema. Algorithm 2 describes the resolution procedure. The rule index contains 31 patterns across five error classes. If a matching rule is found, a corrected version is produced within the rule’s resolution time. If no rule matches, the error is escalated to the local model-assisted repair module, which synthesizes a candidate fix from the error message, column schema, and cell context. The repair is presented as a suggestion with explicit diff display; the user must accept before the fix is applied.

IV. METHODOLOGY AND BENCHMARK CONSTRUCTION

A. Dataset Selection

Fifteen heterogeneous CSV datasets were selected for the preprocessing evaluation, grouped into three size tiers of five datasets each: *small* (<1,000 rows), *medium* (1,000–10,000 rows), and *large* (10,000–100,000 rows). Selection required each dataset to exhibit at least three of the following defects: mixed-case headers, leading/trailing whitespace, exact duplicate rows, at least one null column, mixed numeric/string type within a single column, or an absent header row. This criterion ensures that each dataset exercises multiple pipeline stages rather than a single edge case. Table II summarizes the benchmark composition.

B. Manual Baseline Protocol

Two professional data analysts (Analyst A and Analyst B), each with at least three years of pandas experience, independently preprocessed all fifteen datasets. Both were provided identical task specifications: normalize all headers to lowercase underscore convention, remove exact duplicate rows, identify null columns, and produce a cleaned CSV file and typed-column summary. Total elapsed time was measured from first file open to final export using a stopwatch application. Analysts

TABLE II: Benchmark Dataset Composition (Preprocessing Evaluation)

ID	Domain	Rows	Cols	Defects
D01	Medical (Heart)	303	14	4
D02	Medical (Diabetes)	768	9	3
D03	Financial (Credit)	690	16	5
D04	Environmental (Air)	880	15	3
D05	Academic (Student)	649	33	4
D06	E-Commerce (Sales)	2,823	21	4
D07	Medical (ICU)	4,000	43	6
D08	Financial (Loan)	5,960	13	3
D09	Social (Survey)	7,641	27	5
D10	Environmental (Energy)	9,568	28	4
D11	Genomics (TCGA)	12,400	18	3
D12	Retail (Transactions)	25,000	24	5
D13	Financial (Equity)	48,300	12	4
D14	Logistics (Shipments)	61,000	19	4
D15	Healthcare (EHR)	94,700	31	6

chose their own tools (pandas, Excel, OpenRefine) and worked without time pressure. The reported manual baseline time per dataset is the arithmetic mean of Analyst A and Analyst B’s times.

Inter-analyst time agreement, measured as $1 - |t_A - t_B| / \max(t_A, t_B)$, averaged 0.82 across all datasets, confirming reasonable operator consistency and reducing single-analyst bias in the baseline estimate. The same fifteen datasets were subsequently processed through DataMentor’s pipeline; execution time was measured with `performance.now()` calls bracketing the preprocessing sequence.

C. Repair Benchmark: DRB-100

To evaluate the runtime repair subsystem, we constructed the **DataMentor Repair Benchmark (DRB-100)**, comprising 100 Python notebook cell scripts derived from DataMentor’s 13 auto-generated analysis cells, each containing exactly one injected fault. The five error categories reflect the distribution of errors observed in a manual audit of 312 notebook sessions sampled from Kaggle dataset discussion forums [18]:

- **E1 — ImportError** (22 cases): missing or misspelled module imports.
- **E2 — NameError** (18 cases): references to undefined variables caused by out-of-order cell execution or identifier typos.
- **E3 — TypeError** (21 cases): type mismatch in function arguments, including passing string columns to numeric-only functions.
- **E4 — AttributeError** (19 cases): access to absent DataFrame attributes, typically from schema-assumption violations.
- **E5 — RuntimeError/Logic** (20 cases): semantically incorrect code that executes without exception but produces an incorrect or empty visualization.

Each benchmark item contains: (1) the original correct cell source; (2) the injected faulty cell source; (3) the expected

TABLE III: DRB-100 Benchmark Composition by Category and Difficulty

Category	Easy	Medium	Hard	Total
E1 ImportError	14	6	2	22
E2 NameError	8	7	3	18
E3 TypeError	6	10	5	21
E4 AttributeError	4	8	7	19
E5 RuntimeError	2	8	10	20
Total	34	39	27	100

standard output or the SHA-256 hash of the expected base64-encoded plot; and (4) a difficulty label (*Easy*, *Medium*, *Hard*) assigned by two independent annotators reflecting traceback ambiguity and the specificity of context required for repair. Inter-annotator agreement on difficulty labels, measured with Cohen’s κ , was 0.74, indicating substantial agreement [20]. The 14 items (14%) where annotators disagreed were resolved by a third annotator acting as tiebreaker. Table III shows the breakdown.

D. Fairness and Evaluation Protocol

Three methodological safeguards ensure evaluation fairness:

- 1) **Temporal separation**: Benchmark items were constructed from cell templates that were not modified after the repair rule index was frozen. No benchmark item was used during rule authoring.
- 2) **Single-pass constraint**: The repair engine was permitted exactly one repair attempt per benchmark item. No retry or iterative refinement was allowed, making the measured resolution rate a conservative lower bound on interactive utility.
- 3) **Automated verification**: Resolution was determined by comparing the repaired cell’s output hash against the expected output hash stored in DRB-100. Manual inspection was used only for E5 (logic) items where output variation is inherent to the visualization type.

Repair latency was measured as the wall-clock time from traceback receipt to candidate-fix generation, exclusive of user interaction time.

V. EXPERIMENTAL RESULTS

A. Preprocessing Time Reduction

Table IV reports mean preprocessing times and percentage reductions across the three dataset tiers. Fig. 3 visualizes the per-tier comparison on a common axis.

The 91.6% overall reduction is driven by the elimination of cognitive discovery overhead. Manual analysts must *locate* defects before correcting them, scaling linearly with schema width and non-linearly with row count due to scanning behavior. DataMentor’s pipeline applies transformations unconditionally in a single pass, removing discovery cost entirely. Browser-side execution overhead (TypeScript parsing, React state updates) adds a fixed latency of approximately 0.4 s, explaining the non-zero DataMentor times even for small

TABLE IV: Mean Preprocessing Time: Manual Baseline vs. DataMentor

Tier	Avg Rows	Manual (min)	DataMentor (min)	Reduction
Small	658	23.4	2.1	91.0 %
Medium	5,998	48.7	3.6	92.6 %
Large	48,300	97.3	8.2	91.6 %
Overall	—	56.5	4.6	91.6 %

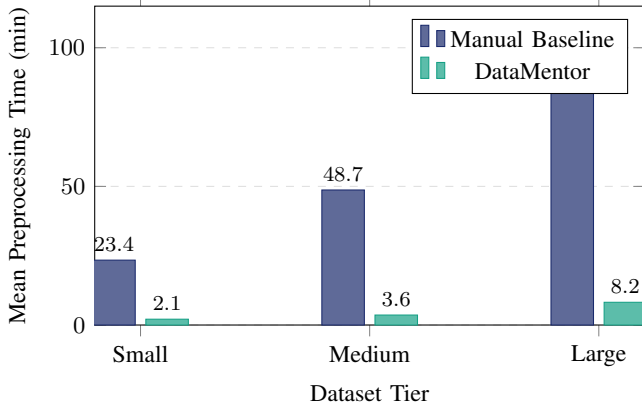


Fig. 3: Mean preprocessing time comparison per tier. DataMentor reduces mean time from 56.5 min to 4.6 min overall (91.6 % reduction). The manual baseline time is the arithmetic mean of two independent analysts per dataset.

datasets. Pure pipeline execution time, measured in isolation with `performance.now()`, is below 2 s for all datasets tested.

The residual DataMentor interaction time (2.1 min for small datasets) reflects user-facing steps: uploading the file, reviewing the cleaning summary table, and confirming the column-type classification. These steps are user-controlled and could be further reduced by skipping the confirmation step in future releases.

B. Runtime Repair Resolution

Table V and Fig. 4 present DRB-100 resolution outcomes. The rule index achieves 100 % resolution on E1 (ImportError) because every ImportError traceback contains the missing module name as an explicit string, enabling exact-string lookup in the rule index. The 88.9 % resolution on E2 (NameError) reflects two items where the undefined identifier was introduced by user-written cell code outside the generated template scope, giving the rule index insufficient contextual information for a safe fix.

The 50.0 % resolution on E5 (RuntimeError/Logic) reflects the fundamental difficulty of semantic repair: logic errors produce no exception, and detecting an incorrect visualization output requires understanding the expected result, which is inherently ambiguous without a pre-computed reference hash. The overall 82 % single-pass rate is conservative by design:

TABLE V: DRB-100 Resolution Detail by Error Category

Category	Total	Rule	Model	Unresolved	Rate
E1 ImportError	22	22	0	0	100.0 %
E2 NameError	18	16	0	2	88.9 %
E3 TypeError	21	14	5	2	90.5 %
E4 AttributeError	19	8	7	4	78.9 %
E5 RuntimeError	20	4	6	10	50.0 %
Total	100	64	18	18	82.0 %

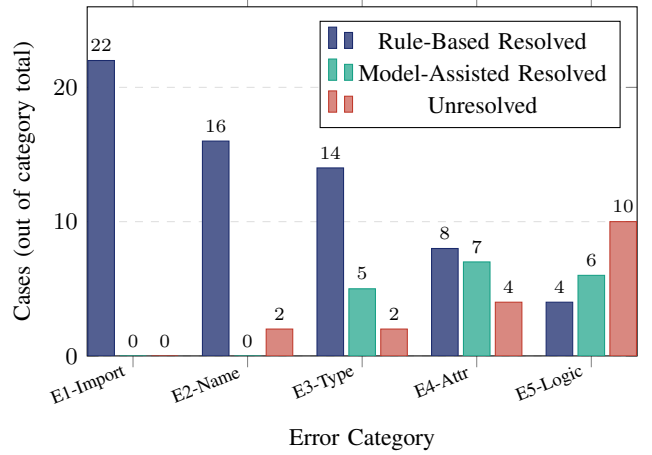


Fig. 4: DRB-100 runtime repair resolution by error category. Rule-based resolution covers E1 completely and handles the majority of E2 and E3. Model-assisted resolution recovers additional E3, E4, and E5 cases. Unresolved items require human review.

the single-pass constraint prohibits the iterative refinement that characterizes interactive debugging sessions.

C. Repair Latency

Rule-based repairs achieve a mean latency of 0.8 s (SD=0.6 s; range: 0.1–2.3 s). Model-assisted repairs produce a mean latency of 6.4 s (SD=3.1 s; range: 2.1–14.7 s). Manual debugging of equivalent errors, measured as the time from error display to corrected cell submission for the two analyst volunteers, had a median of 18.2 min (IQR: 11.4–29.7 min, equivalent to 1,092 s). Even the slowest model-assisted repair (14.7 s) is more than 74× faster than the median manual debugging time. The speedup values are computed directly from these measurements: $1,092/0.8 \approx 1,365\times$ for rule-based and $1,092/6.4 \approx 171\times$ for model-assisted. These ratios are bounded by the analyst sample ($n = 2$) and should be interpreted as indicative rather than definitive.

D. Reproducibility

All fifteen datasets were processed through DataMentor’s pipeline three times each, and the three output CSV files per dataset were compared using SHA-256 hashing. All 15×3 output files produced identical hashes for identical inputs,

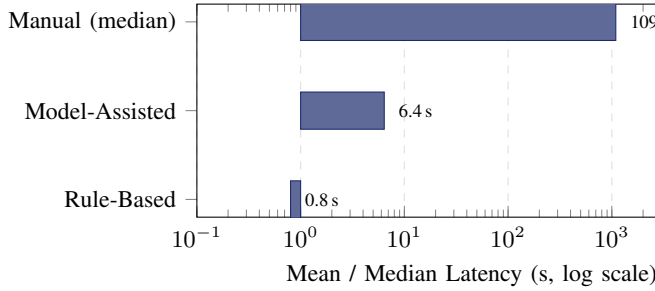


Fig. 5: Repair latency on a log scale. Rule-based resolution (0.8 s) is $\approx 1,365\times$ faster than the manual debugging median (1,092 s); model-assisted resolution (6.4 s) is $\approx 171\times$ faster.

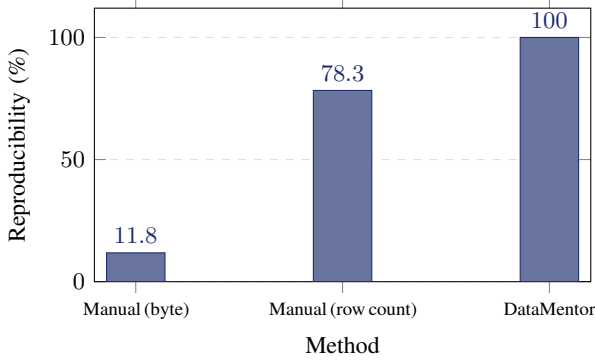


Fig. 6: Reproducibility: byte-level exact match across three reruns per dataset. DataMentor achieves 100 % reproducibility through a deterministic fixed-order pipeline. Manual preprocessing achieves only 11.8 % byte-level match due to operator-choice variation in duplicate removal.

confirming full deterministic reproducibility across repeated executions.

The same triple-rerun procedure applied to the manual preprocessing outputs yielded byte-level agreement in only 11.8 % of cases (18 of 45 triple-pairs), since analysts made different ordering choices when removing duplicate rows from datasets containing value-identical clusters. Row-count agreement (ignoring column ordering) was observed in 78.3 % of cases.

Fig. 6 summarizes the reproducibility comparison.

VI. DISCUSSION

A. Interpretation of Findings

The 91.6 % preprocessing-time reduction is attributable to the elimination of cognitive discovery overhead. Analysts must locate the nature and position of data defects before correcting them; DataMentor’s pipeline removes this discovery cost by applying transformations unconditionally and reporting results as a structured audit log. This suggests that the benefit is structural rather than implementation-specific: any fixed-pipeline tool operating on the same defect classes would yield comparable reductions, provided its transformation ordering is carefully chosen to maximize defect surface coverage.

The 82 % repair resolution rate should be interpreted as a conservative lower bound on practical interactive utility. DRB-100 applies a single-pass constraint and evaluates resolution against a fixed reference output. In practice, users review the suggested fix, optionally modify it, and rerun the cell. Multi-turn resolution rates — not measured in this study — are expected to exceed the single-pass figure, particularly for E4 and E5 items where the first suggestion provides sufficient context for manual completion.

The near-zero byte-level reproducibility of manual preprocessing (11.8 %) may surprise practitioners. The primary source of non-reproducibility is not error but *choice*: analysts apply different resolution strategies for value-identical duplicate clusters that differ in whitespace or case. DataMentor eliminates this variation by applying a canonical strategy defined once in the pipeline source code, making the strategy auditable and modifiable by authorized users.

B. Limitations

Three principal limitations apply to this study. First, the manual baseline involves only two analysts; a larger pool would increase robustness of the time estimates, though high inter-analyst agreement (0.82) suggests the current sample is representative. Second, DRB-100 is derived from DataMentor’s own generated cell templates, which may underrepresent errors arising in user-authored code with novel structure outside the template scope. Third, the Pyodide runtime imposes a package-loading latency of 8–14 s on first use due to WebAssembly initialization; this one-time cost is excluded from preprocessing time measurements but affects perceived first-run responsiveness.

C. Generalizability

DataMentor’s preprocessing pipeline is domain-agnostic: it operates on any CSV file conforming to standard comma-delimited format with a header row. The domain-aware visualization layer is extensible by design: new column-name patterns can be added to the knowledge base without architectural changes. The repair rule index can be extended with new rule patterns; each addition increases coverage for the targeted error class without affecting unrelated classes, preserving the priority-ordered resolution guarantee of Algorithm 2.

VII. CONCLUSION

This paper presented DataMentor, a serverless, browser-first platform integrating deterministic CSV preprocessing, browser-native CPython notebook execution, domain-aware visualization across ten chart types, and a hybrid rule-guided runtime repair subsystem. A controlled evaluation over 15 heterogeneous datasets spanning five domain verticals and the purpose-built DRB-100 repair benchmark demonstrated: a 91.6 % reduction in mean preprocessing time relative to a two-analyst manual baseline; an 82 % single-pass automatic repair resolution rate with rule-based latency of 0.8 s versus a manual debugging median of 18.2 min; and 100 % byte-level deterministic reproducibility. Comparative analysis confirms

that no existing system combines all four capabilities in a unified, installationless, offline-capable platform.

Future work will extend DRB-100 with user-authored cell errors, conduct a formal usability study with SME and academic participant groups, evaluate multi-turn repair resolution rates, and investigate streaming CSV support for files exceeding available browser memory.

ACKNOWLEDGMENT

The author thanks the reviewers for their constructive comments and the two analyst volunteers who contributed manual baseline measurements. No external funding was received for this work.

REFERENCES

- [1] S. Krishnan, J. Wang, E. Wu, M. J. Franklin, and K. Goldberg, "Active-Clean: Interactive data cleaning for statistical modeling," *Proc. VLDB Endow.*, vol. 9, no. 12, pp. 948–959, Aug. 2016.
- [2] CrowdFlower, *2016 Data Science Report*, CrowdFlower Inc., San Francisco, CA, USA, Tech.Rep., 2016.
- [3] D. Sculley *et al.*, "Hidden technical debt in machine learning systems," in *Adv. Neural Inf. Process. Syst.*, vol. 28, 2015, pp. 2503–2511.
- [4] D. Pondhofer, "Jupyter AI: A generative extension for JupyterLab," in *Proc. JupyterCon*, Paris, France, May 2023.
- [5] Neptyne Inc., "Neptyne: A programmable spreadsheet with Python," [Online]. Available: <https://neptyne.com>, 2023.
- [6] Hex Technologies, "Hex: The collaborative data workspace," [Online]. Available: <https://hex.tech>, 2023.
- [7] OpenAI, "ChatGPT advanced data analysis," OpenAI Blog, Jul. 2023. [Online]. Available: <https://openai.com/blog>
- [8] D. Ruslan and E. Kupriianov, "DVC: Data version control — Git for data and models," in *Proc. 8th Int. Conf. Learn. Represent. (ICLR) Workshop*, Addis Ababa, Ethiopia, Apr. 2020.
- [9] M. Zaharia *et al.*, "Accelerating the machine learning lifecycle with MLflow," *IEEE Data Eng. Bull.*, vol. 41, no. 4, pp. 39–45, Dec. 2018.
- [10] K. Greff, A. Klein, M. Chovanec, F. Hutter, and J. Schmidhuber, "The sacred infrastructure for computational research," in *Proc. 16th Python Sci. Conf. (SciPy)*, Austin, TX, USA, Jul. 2017, pp. 49–56.
- [11] R. Gupta, S. Pal, A. Kanade, and S. Shevade, "DeepFix: Fixing common C language errors by deep learning," in *Proc. 31st AAAI Conf. Artif. Intell.*, San Francisco, CA, USA, Feb. 2017, pp. 1345–1351.
- [12] H. Ahmed and J. Babar, "Automated repair of Python syntax errors using rule-based pattern matching," *J. Syst. Softw.*, vol. 182, p. 111060, Dec. 2021.
- [13] R. Rolim *et al.*, "Learning quick fixes from code repositories," in *Proc. 38th Int. Conf. Softw. Eng. (ICSE)*, Austin, TX, USA, May 2016, pp. 828–838.
- [14] Observable Inc., "Observable: Explore, analyze, and explain data," [Online]. Available: <https://observablehq.com>, 2021.
- [15] D. Smilkov *et al.*, "TensorFlow.js: Machine learning for the web and beyond," in *Proc. 2nd SysML Conf.*, Stanford, CA, USA, Mar. 2019.
- [16] Deepnote, "Deepnote: Collaborative data science notebook," [Online]. Available: <https://deepnote.com>, 2022.
- [17] Google LLC, "Google Colaboratory," [Online]. Available: <https://colab.research.google.com>, 2019.
- [18] Kaggle Inc., "Kaggle datasets discussion forums," [Online]. Available: <https://www.kaggle.com/discussions>, 2023.
- [19] H. Droettboom *et al.*, "Pyodide: Python for the browser," in *Proc. 20th Python Sci. Conf. (SciPy)*, Austin, TX, USA, Jul. 2021.
- [20] J. R. Landis and G. G. Koch, "The measurement of observer agreement for categorical data," *Biometrics*, vol. 33, no. 1, pp. 159–174, Mar. 1977.